Apache Pekko for Distributed Systems

Pekko Artery Remoting and Pekko Cluster: Introduction

Nicolas Farabegoli nicolas.farabegoli@unibo.it

10 May 2026

Department of Computer Science and Engineering (DISI) — University of Bologna

Agenda

Today's Topics

- **Cluster Concepts**: Membership, gossip convergence, and failure detection
- **Pekko Remote Artery**: Location transparency, configuration, and serialization
- **Pekko Clustering**: Cluster membership, gossiping, and failure detection
- **Advanced Facilities**: Receptionist, Group Router, Distributed Data, and Sharding

Cluster Concepts

Distributed Membership

What Pekko Cluster models

Pekko Cluster lets one application span several actor systems, each running as a logical **node**. A node is identified by `hostname:port:uid`, so several nodes may even run on the same physical machine.

Decentralized

Membership is peer-to-peer: no permanent master, no central registry, and no single bottleneck for cluster state.

What is tracked

The cluster tracks members, their lifecycle state, and whether each member is currently `reachable` or `unreachable`.

Gossip and Convergence

How nodes agree

Cluster state is disseminated by **gossip**[1]: nodes periodically exchange their view of membership with other nodes, preferring peers that have not seen the latest state.

Vector clocks

Each state update carries version information, allowing nodes to detect older, newer, or conflicting membership views and merge them.

Seen set

A state is **converged** when **every reachable node has observed the current version**. Until convergence, membership transitions wait.

[1] https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf

Failure Detection

Failure is suspicion, not certainty

Pekko uses a Phi Accrual Failure Detector: heartbeat timing is interpreted as a suspicion level, which can be tuned for the deployment environment.

i Spread by gossip

When one monitor marks a node `unreachable`, that information is propagated through the cluster. The node may later become `reachable` again.

! Operational consequence

Gossip convergence cannot complete while members are unreachable. They must recover, or be downed and removed, before the leader can advance membership.

Split Brain Resolver

The Problem: Network Partitions

Network partitions, machine crashes, and unresponsiveness are fundamentally indistinguishable to the observer. A naive auto-downing approach based solely on timeouts can lead to a **split brain** (two disconnected clusters forming).

! Consequences of a Split Brain

Fatal for **Cluster Singleton**, **Cluster Sharding**, and **Pekko Persistence** (multiple active instances and concurrent writers corrupting data).

i The Solution

The **Split Brain Resolver (SBR)** ensures both sides of a partition make the same deterministic decision on which part survives.

Split Brain Resolver Strategies

Configuration

```
1 pekko.cluster.downing-provider-class = "o.a.p.c.sbr.SplitBrainResolverProvider"
2 pekko.cluster.split-brain-resolver.active-strategy = keep-majority
```

Common Strategies

- **Keep Majority**: keeps partition with most nodes.
- **Static Quorum**: requires minimum configured nodes.
- **Keep Oldest**: keeps partition with oldest node.

Other Strategies

- **Down All**: safe alternative for unstable networks.
- **Lease**: relies on external arbiter (e.g. Kubernetes).

Leader and Joining

The Leader

- **Deterministic:** Automatically assigned after gossip convergence. It is **not** an elected master.
- **Duties:** Promotes nodes from `joining` to `up` and completes the removal of `exiting`/`down` nodes.

Seed Nodes

- **Entry Points:** Serve as the initial contact addresses for new members joining the cluster.
- **Peer-to-Peer:** Once the cluster is running, they hold no special status or central coordination role.

Leader and Joining

No Single Point of Failure

Pekko Cluster is entirely **peer-to-peer**. The Leader and Seed Nodes facilitate the membership lifecycle, but they do not introduce central bottlenecks once the cluster is established.

Pekko Remote Artery

Artery Overview

What Artery is

Artery is the remoting subsystem by which actor systems on different nodes **talk to each other**. It replaced classic remoting and keeps the actor API **location-transparent**.

Recommended usage

In practice, prefer Pekko Cluster, or higher-level protocols such as HTTP and gRPC, instead of using remoting directly.

Peer-to-peer

Remoting is not server-client: any remoting-enabled system can connect to any other one if it knows the ActorRef.

Selecting a Transport

Three transport choices

- `tcp`: default and simplest choice
- `tls-tcp`: TCP with encryption
- `aeron-udp`: high throughput and low latency, but more operationally demanding

Practical guidance

Use `tcp` unless you need TLS or Aeron-specific performance. Switching the transport protocol later is not a rolling update.

Configuration

We can configure Artery Remoting in application.conf:

application.conf

```
pekko.actor.provider = cluster
pekko.remote.artery {
  transport = tcp
  canonical.hostname = "127.0.0.1"
  canonical.port = 25520
}
```

Canonical address

The canonical host and port are the globally reachable address that other systems use to connect back. In NAT, Docker, or Kubernetes setups, separate the external canonical address from the local bind address. See the documentation for details.

Acquiring references to remote actors

To communicate with (remote) actors, you need their `ActorRef`. You can get it by:

In a message

Include the `ActorRef` in the message payload, e.g. `Reply(..., ref: ActorRef[T])`. But the first message?

Via actor selection

Use `actorSelection(path)` to look up an actor by its path. This is not location-transparent and should be used with care.

! Be careful

Both methods require you to know the remote actor's path or have it sent to you, which can break location transparency and tightens coupling between actors.

Serialization

Why serialization matters

Messages between actors in the same JVM are passed by reference, but once a message leaves the JVM it must be turned into bytes and reconstructed on the other side.

Recommended choices

- **Jackson** is the recommended default for application messages
- **Protocol Buffers** are a good fit when you want tighter schema control

Core idea

Pekko separates:

- the **serializer implementation**
- the **binding** from message type to serializer

Serialization configuration

application.conf

```
pekko.actor.serializers {
  jackson-cbor = "org.apache.pekko.serialization.jackson.JacksonCborSerializer"
  proto = "org.apache.pekko.remote.serialization.ProtobufSerializer"
}

pekko.actor.serialization-bindings {
  "io.github.nicolasfara.es01.cluster.CborSerializable" = jackson-cbor
  "com.google.protobuf.Message" = proto
}
```

```
libraryDependencies += "org.apache.pekko" %% "pekko-serialization-jackson" % PekkoVersion
```

Serialization binding rule

Binding rule

Bind a trait, interface, or abstract base class rather than each concrete message class. If more than one binding matches, Pekko uses the most specific one and warns about ambiguous cases.

Remote Security

Protect remoting

- Prefer `tls-tcp` when messages must be encrypted
- Use mutual authentication between peers
- Do not expose Artery directly on an untrusted network

! Untrusted mode

`pekko.remote.artery.untrusted-mode = on` blocks remote deployment, remote DeathWatch, system-stop style messages, and messages marked `PossiblyHarmful`.

Remote Watch and Quarantine

- **Remote watch:** You can watch remote actors just like local actors. A failure detector uses heartbeats to generate `Terminated`.
- System messages for death watch are delivered with "exactly once" guarantee.
- If a system message cannot be delivered, the destination enters the **quarantined** state.
- The only way to recover from quarantine is to restart the actor system.

Pekko Clustering

Pekko Cluster: basic usage (1/2)

```
libraryDependencies ++= Seq(
  "org.apache.pekko" %% "pekko-cluster-typed" % PekkoVersion,
  "org.apache.pekko" %% "pekko-serialization-jackson" % PekkoVersion
)
```

The Cluster extension gives you access to management tasks such as Joining, Leaving and Downing and subscription of cluster membership events such as MemberUp, MemberRemoved, and UnreachableMember.

```
// Access the Cluster extension on a node
val cluster = Cluster(system)
```

Pekko Cluster: extension references

Key references on the Cluster extension

- manager: an ActorRef[ClusterCommand] (e.g., Join, Leave, Down)
- subscriptions: an ActorRef[ClusterStateSubscription]
- state: the current CurrentClusterState

Pekko Cluster: basic usage (2/2)

application.conf

```
pekko.actor.provider = "cluster"
pekko.remote.artery {
  canonical {
    hostname = "127.0.0.1"
    port = 2551
  }
}
pekko.cluster.seed-nodes = [
  "pekko://ClusterSystem@127.0.0.1:2551",
  "pekko://ClusterSystem@127.0.0.1:2552"
]
```

Cluster Membership: Joining

Joining through **seed nodes** (point of contact for new nodes that join the cluster):

1. **Join configured seed nodes:**
   pekko.cluster.seed-nodes=["pekko://Sys@host1:2551", ...]
   The first seed **must be started first** to allow other seeds to join!
2. **Join seed nodes programmatically:**

```
1 val seedNodes: List[Address] = // discover in some way
2 Cluster(system).manager ! JoinSeedNodes(seedNodes)
```

3. **Join automatically:** via Cluster Bootstrap (usually with Kubernetes or similar).

Joining a cluster programmatically

Without using seed nodes: cluster.manager ! Join(cluster.selfMember.address)

Cluster Membership: Leaving & Subscriptions

Leaving a cluster

- **Programmatically:** `cluster.manager ! Leave(address)`
- **Graceful exit:** via Coordinated Shutdown (e.g. `sys.terminate()`)
- **Non-graceful exit:** E.g. in case of abrupt termination, the node will be detected as unreachable by other nodes and removed after Downing.

Subscriptions

Receive cluster state changes, e.g. to be notified of a node leaving the cluster:

```
1  val subscriber: ActorRef[MemberEvent] = ...
2  cluster.subscriptions ! Subscribe(
3    subscriber, classOf[MemberEvent]
4  )
```

Node roles

Motivation

Not all nodes of a cluster need to perform the same function. Choosing which actors to start on each node can take roles into account to properly distribute responsibilities.

Configuration: Config key `pekko.cluster.roles`

Getting role info: Role info is included in membership information (cf. `MemberEvent`). For the own node, `cluster.selfMember.hasRole(r)`.

```
1 val selfMember = Cluster(context.system).selfMember
2 if (selfMember.hasRole("backend")) {
3   context.spawn(Backend(), "back")
4 } else { /* spawn frontend or other roles */ }
```

Pekko Cluster facilities: discovery

Find actors by capability

- `ServiceKey[T]` names a protocol, not a location
- The receptionist keeps a cluster-wide registry of live services
- Lookups and subscriptions return reachable `ActorRef`s

Replicate small shared state

- Distributed Data stores values as CRDTs
- Local updates spread through gossip
- Conflicts are resolved by the data type
- Used by receptionist, and available to applications

Route to whatever is available

A group router is created from a `ServiceKey`: it watches receptionist listings and routes messages to the currently reachable routees.

Pekko Cluster facilities: placement

One logical owner

Cluster singleton keeps exactly one active actor for a role in the whole cluster.

Use it for coordinators, schedulers, and other responsibilities that must not be duplicated.

Many logical owners

Cluster sharding spreads entities across nodes and lets clients address them by logical ID.

The ShardRegion extracts entity IDs; the coordinator decides where shards live.

i Rule of thumb

Singleton is for **one** cluster-wide responsibility. Sharding is for **many** independent entities that should move and rebalance with the cluster.

Receptionist and Service Keys

When an actor needs to be discovered by another actor, the recommended approach is to use the **Receptionist** and **Service Keys**:

How discovery works

- Register a service actor under a typed `ServiceKey[T]`
- Other actors query the receptionist through messages, not direct lookups
- A `Listing` reply contains the current `Set[ActorRef[T]]` for that key

Dynamic registry

- `Receptionist.Register(key, ref)` makes an actor discoverable
- `Receptionist.Find(...)` gives a point-in-time snapshot
- `Receptionist.Subscribe(...)` pushes the first listing and later changes

Receptionist and Service Keys: API

```
1 val PingServiceKey = ServiceKey[Ping]("pingService")
2
3 context.system.receptionist ! Receptionist.Register(PingServiceKey, context.self)
4 context.system.receptionist ! Receptionist.Find(PingServiceKey, listingAdapter)
5 context.system.receptionist ! Receptionist.Subscribe(PingServiceKey, context.self)
```

Lifecycle

Several actors may share the same key. Entries disappear when an actor stops, is deregistered, or its node is removed from the cluster.

Cluster Receptionist

Cluster semantics

- Registrations on one node **appear in the receptionist of the other cluster** nodes
- State is propagated via distributed data
- Convergence is eventual: nodes reach the same service set per `ServiceKey`

i Reachability-aware listings

- `Find` and `Subscribe` only return **reachable** service instances
- Unreachable ones are excluded
- The full set can still be inspected through `Listing.allServiceInstances`

! Important constraints

Cluster receptionist is **great for initial contact** and loose discovery, but all cross-node messages must be serializable and the receptionist **is not meant** for unlimited scale or very high service churn.

Routers

Why routers exist

A router is **itself an actor**: you send one message to the router, and it forwards that message to one routee chosen from a set of actors able to handle the same protocol.

Pool router

- Created from a `Behavior[T]`
- Spawns a fixed number of local child
- Best when you want parallel workers inside one actor system

Group router

- Created from a `ServiceKey[T]`
- Uses the receptionist to discover routees
- Can route to actors on other reachable cluster nodes

Routers: distributed takeaway

Distributed takeaway

For clustered applications, the interesting one is the **group router**: it composes naturally with receptionist-based discovery and keeps senders decoupled from concrete actor locations.

Pool Router

Semantics

- `Routers.pool(n)(behavior)` creates `n` routee children
- Routees are **always local**: a pool does not distribute work across the cluster
- If a child stops, the router removes it from the pool

Operational notes

- Supervise the worker behavior if failures should restart it
- The default strategy is **round robin**
- A pool can also recognize special messages that should be **broadcast** to all routees

```
1 val pool = Routers.pool(poolSize = 4) {
2   Behaviors.supervise(Worker()).onFailure[Exception](SupervisorStrategy.restart)
3 }
4 val router = ctx.spawn(pool, "worker-pool")
5 router ! Worker.DoLog("msg")
```

Group Router

How it works

- Register workers under a ServiceKey[T]
- Build the router with Routers.group(serviceKey)
- The router subscribes to receptionist listings and forwards messages to discovered routees

Cluster behavior

- Reachable routees on **any cluster node** may be selected
- Membership is **eventually consistent** because discovery relies on receptionist
- On startup, the router stashes messages until it receives its first listing

```
1 val serviceKey = ServiceKey[Worker.Command]("log-worker")
2 ctx.system.receptionist ! Receptionist.Register(serviceKey, worker)
3
4 val group = Routers.group(serviceKey)
5 val router = ctx.spawn(group, "worker-group")
6 router ! Worker.DoLog("msg")
```

Group Router: empty listings

! Important edge case

After the first receptionist listing, if the discovered set is empty, the group router drops incoming messages. That makes it great for elastic discovery, but not a substitute for guaranteed delivery or back-pressure.

Routing strategies and limits

Built-in strategies

- **Round robin:** fair rotation across routees; default for pool routers
- **Random:** good when membership changes often; default for group routers
- **Consistent hashing:** same key tends to hit the same routee while membership stays stable

! Performance reality

- More routees help only if the workload and dispatcher can exploit parallelism
- For CPU-bound workers, extra routees beyond available threads seldom help
- The router head processes incoming messages sequentially, so it can become a bottleneck at very high throughput

Routing strategies: when to use what

When to use what

Use a **pool router** for local worker parallelism, a **group router** for cluster-aware service discovery, and consider Cluster Sharding when you need stable key-based routing with rebalancing.

Wrap-up

Acknowledgement

Acknowledgement

The material and the slides are derived from Roberto Casadei and Gianluca Aguzzi's previous seminar on Akka Cluster.

Originally focused on Akka, these slides have been adapted to present the modern Apache Pekko ecosystem.

References

- Apache Pekko Documentation: pekko.apache.org/docs/
- Pekko Remoting: remoting-artery.html
- Pekko Serialization: serialization.html
- Pekko Cluster Specification: typed/cluster-concepts.html
- Pekko Cluster: typed/cluster.html
- Pekko Typed Routers: typed/routers.html